



Polimorfismo

Desarrollo Orientado a Objetos



¿Qué aprenderemos hoy?

01

¿Qué es el polimorfismo?

02

¿Por qué importa el polimorfismo?

03

Tipos de polimorfismo en Java

04

Herencia y Sobrescritura

05

Clase Base Animal

06

Subclases Perro , Gato y Pato

07

Referencia Poliformica

08

Polimorfismo en Colecciones.

09

Métodos que reciben Animal

10

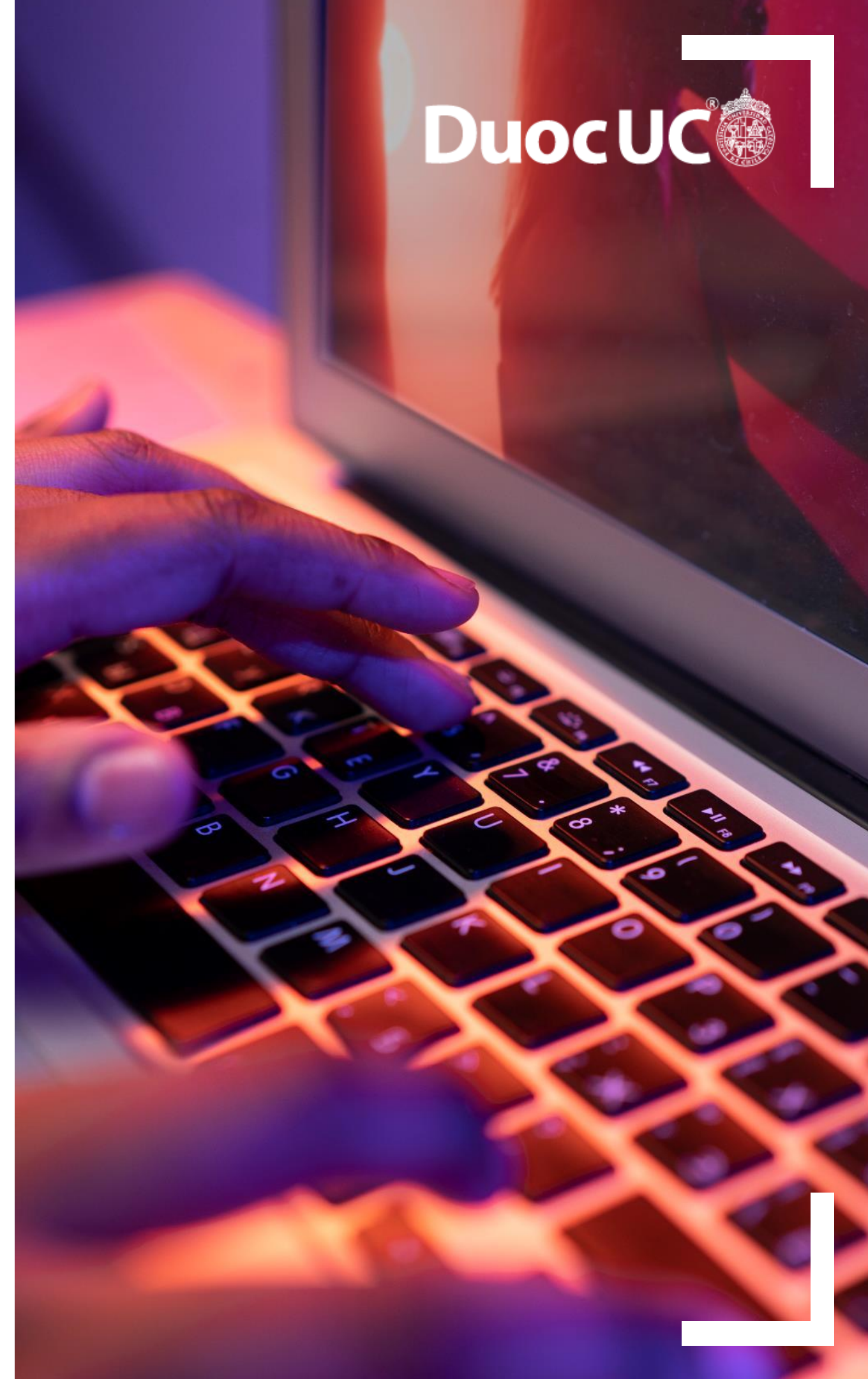
Buenas practicas al utilizar polimorfismo

11

Errores comunes y checklist

12

Conclusiones y Próximos pasos



```
Animal animal1 = new Perro(); Animal animal2 = new Gato(); animal1.hacerSonido(); // Guau animal2.hacerSonido(); // Miau
```

</> Programación Orientada a Objetos

Java

Polimorfismo en Java

Una misma referencia, múltiples comportamientos
con ejemplos de animales.



Referencia Única
Manejo genérico (Animal)



**Comportamiento
Dinámico**
Ejecución específica

 Polimorfismo.java

```
// Referencia tipo Animal,  
// Objeto real tipo Perro  
Animal miAnimal = new Perro();  
// Despacho dinámico: se ejecuta  
// el método de la clase Perro  
miAnimal.hacerSonido();  
// Salida:  
// "¡Guau!"
```



¿Qué es el polimorfismo?

Un mensaje, múltiples comportamientos

💡 Concepto Central

El polimorfismo permite que una referencia de tipo general (como **Animal**) ejecute el comportamiento específico del objeto real al que apunta (como **Perro** o **Gato**).



Referencia Común

Usas una variable del tipo padre para agrupar objetos.



Mismo Mensaje

Llamas al mismo método en todos los objetos.



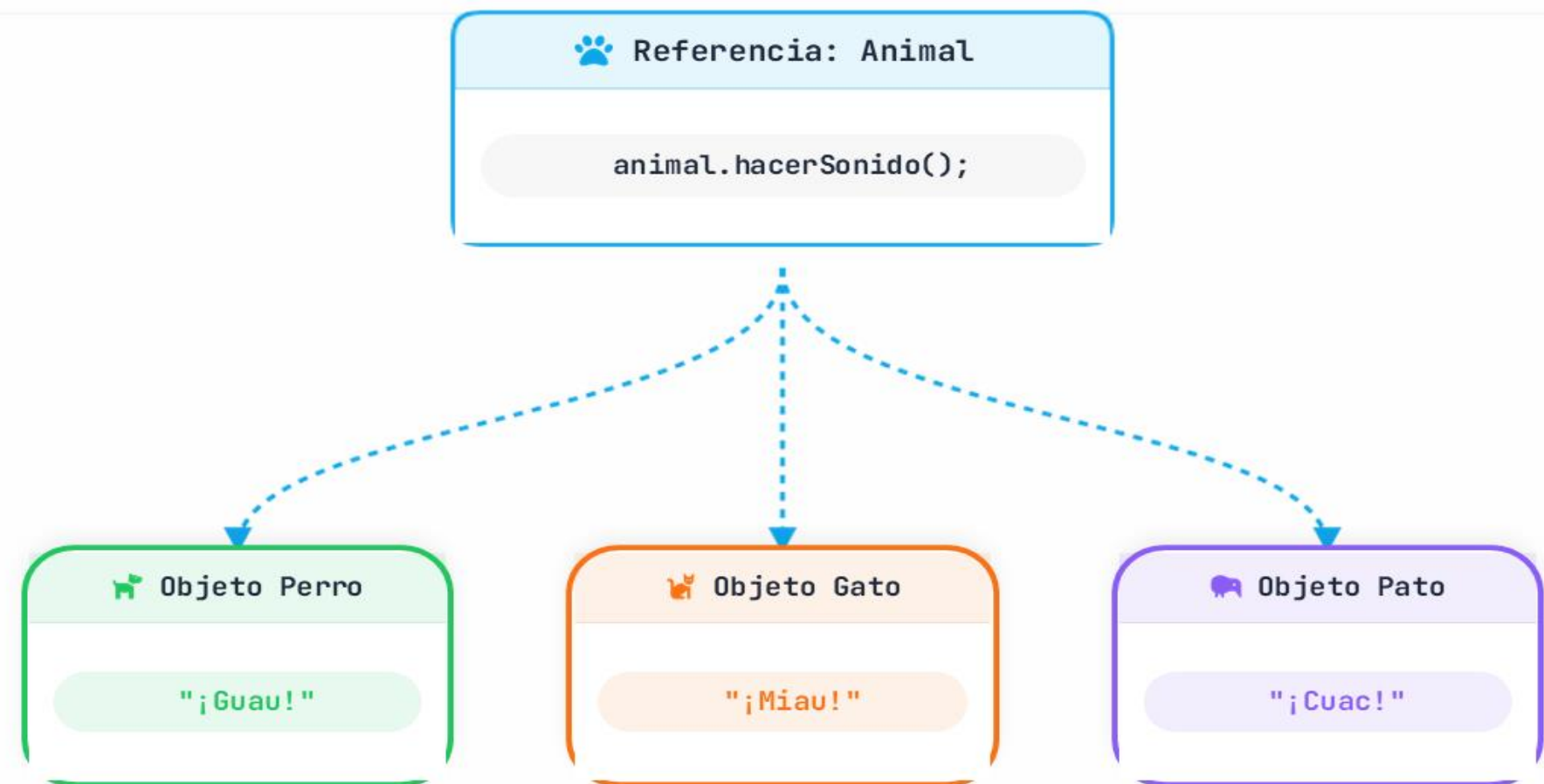
Distinta Respuesta

Cada objeto responde a su manera según su clase real.

En resumen:

📌 Diferentes formas de responder a una misma acción.

El mismo mensaje en acción



¿Por qué importa el polimorfismo?

Flexibilidad y escalabilidad en el diseño de software

★ Beneficios Principales

El polimorfismo permite crear sistemas que pueden evolucionar fácilmente sin modificar el código existente.



Extensibilidad

Agrega nuevas clases (ej. León) sin alterar la lógica central.



Bajo Acoplamiento

El código cliente depende de abstracciones (Animal), no de detalles.



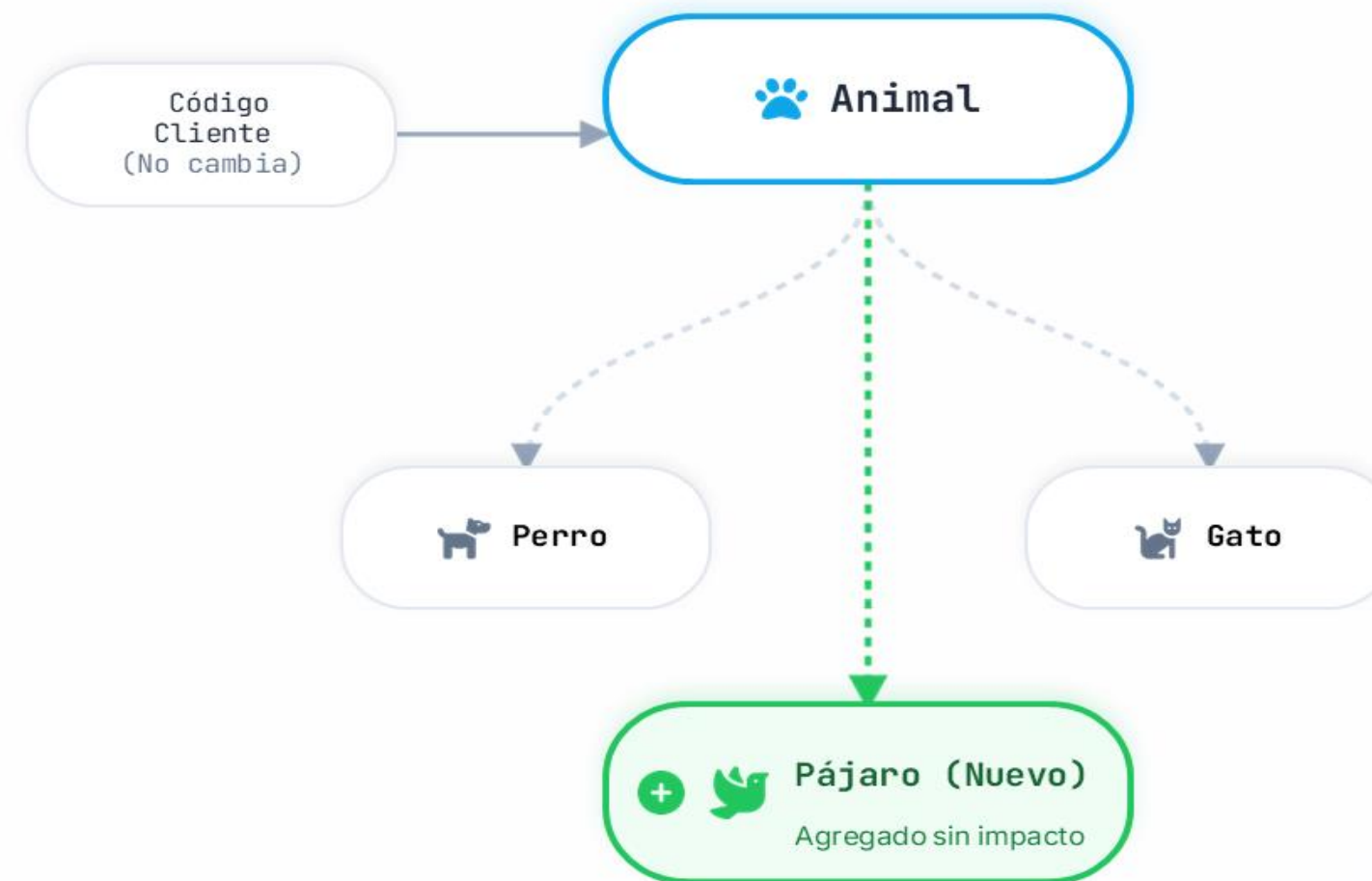
Reutilización

Escribe métodos genéricos que funcionan con múltiples tipos.

Resultado Final:

✓ **Sistemas modulares y mantenibles**

Extensibilidad en Acción



Diseño Base



Contratos Flexibles





Tipos de Polimorfismo en Java

Compilación vs Ejecución

Mecanismos

El polimorfismo en Java se presenta en dos formas principales, dependiendo de cuándo el compilador o la máquina virtual deciden qué método invocar.



Tiempo de Compilación

Conocido como **Sobrecarga (Overloading)**. Varios métodos con el mismo nombre pero distintos parámetros en una misma clase.



Tiempo de Ejecución

Conocido como **Sobrescritura (Overriding)**. Una subclase redefine un método de la superclase. La JVM decide cuál llamar.

Resolución:

La sobrecarga es estática (early binding). La sobrescritura es dinámica (late binding).

Comparación Visual

Sobrecarga (Compilación)

 `class Perro`

```
void alimentar() {  
    print("Come croquetas");  
}
```

```
void alimentar(Carne c) {  
    print("Come carne");  
}
```

Mismo método, diferentes parámetros. El compilador sabe cuál llamar según los argumentos.

Sobrescritura (Ejecución)

`class Animal`

`void hacerSonido()`

Perro

```
@Override  
void hacerSonido()  
"Guau"
```

Gato

```
@Override  
void hacerSonido()  
"Miau"
```

Mismo método, distinta clase. La JVM decide en tiempo de ejecución según el objeto real.



Sobrecarga (Overloading)



Sobrescritura (Overriding)





Herencia y Sobrescritura

La base del polimorfismo dinámico



El rol de @Override

El polimorfismo en Java depende de que las subclases proporcionen su propia implementación para los métodos definidos en la superclase.



Asegura Corrección

El compilador verifica que la firma del método coincida exactamente con la superclase.



Sustituye Comportamiento

Al llamar al método, Java ejecuta la versión más específica en tiempo de ejecución.



Intención Clara

Documenta para otros desarrolladores que el método redefine un comportamiento heredado.

Flujo de Sobrescritura

 **clase Animal**

`void hacerSonido()`

@Override

 **Perro**

`void
hacerSonido()`

"¡Guau!"

@Override

 **Gato**

`void
hacerSonido()`

"¡Miau!"

@Override

 **Pato**

`void
hacerSonido()`

"¡Cuac!"



Polimorfismo Dinámico



Clase base: Animal

Contrato común y estado base para comportamiento polimórfico

Clase Padre (Superclase)

La clase base **Animal** define los atributos comunes y establece el método `hacerSonido()`, el cual está diseñado para ser sobrescrito y actuar de manera polimórfica.



Estado Común

Centraliza propiedades generales como nombre y edad.



Contrato Polimórfico

Provee un método genérico `hacerSonido()` que garantiza una interfaz uniforme.

Beneficio Clave:

- 🔑 Punto único de referencia

Estructura de la Clase Base

class Animal

Atributos (Estado Común)

```
A String nombre    # int edad
```

Método Base (Polimórfico)

```
« public void hacerSonido() {  
  System.out.println("Sonido genérico");  
}
```



Subclases: Perro, Gato y Pato

Especialización y redefinición del comportamiento base

Especialización Obligatoria

Las subclases heredan de **Animal** pero adaptan el comportamiento genérico a su propia naturaleza usando la anotación **@Override**.



Perro

Implementa `hacerSonido()` devolviendo "Guau". Es su forma específica de comunicarse.



Gato

Redefine el mismo método `hacerSonido()` pero con su comportamiento propio: "Miau".



Pato

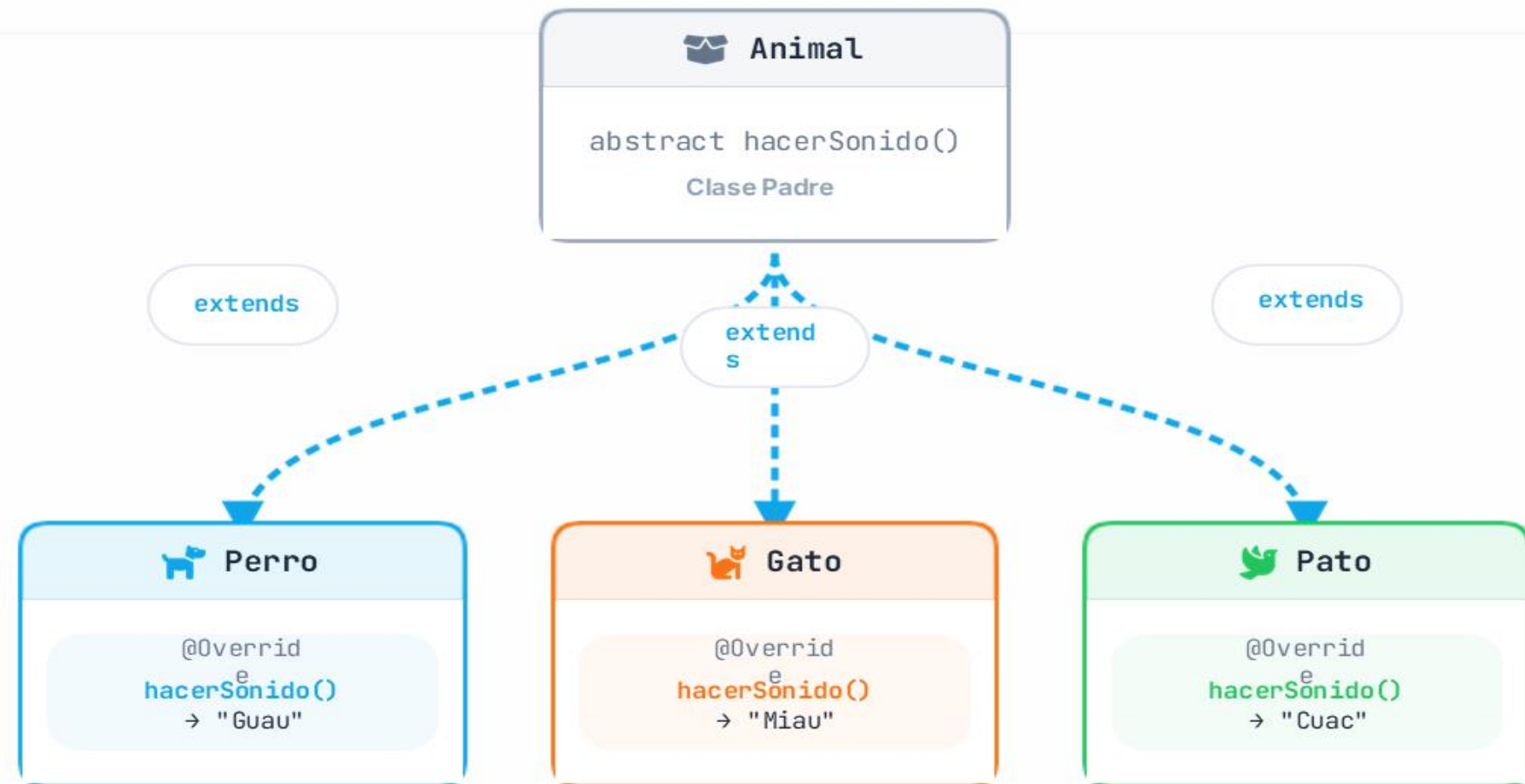
Hereda la estructura base, pero al llamar a `hacerSonido()` el resultado será "Cuac".

Concepto Clave:



Mismo contrato (método), múltiples formas de ejecución

Estructura de Implementación



Referencia Polimórfica

Una variable, múltiples formas en tiempo de ejecución

⚡ Despacho Dinámico

En tiempo de ejecución, Java determina qué implementación de un método llamar basándose en el **tipo real del objeto instanciado**, no en el tipo de la referencia.



Referencia Genérica

Una variable `Animal` permite apuntar a cualquier subclase sin conocer su tipo específico.



Asignación Dinámica

La misma referencia puede reasignarse a objetos `Perro`, `Gato` o `Pato`.



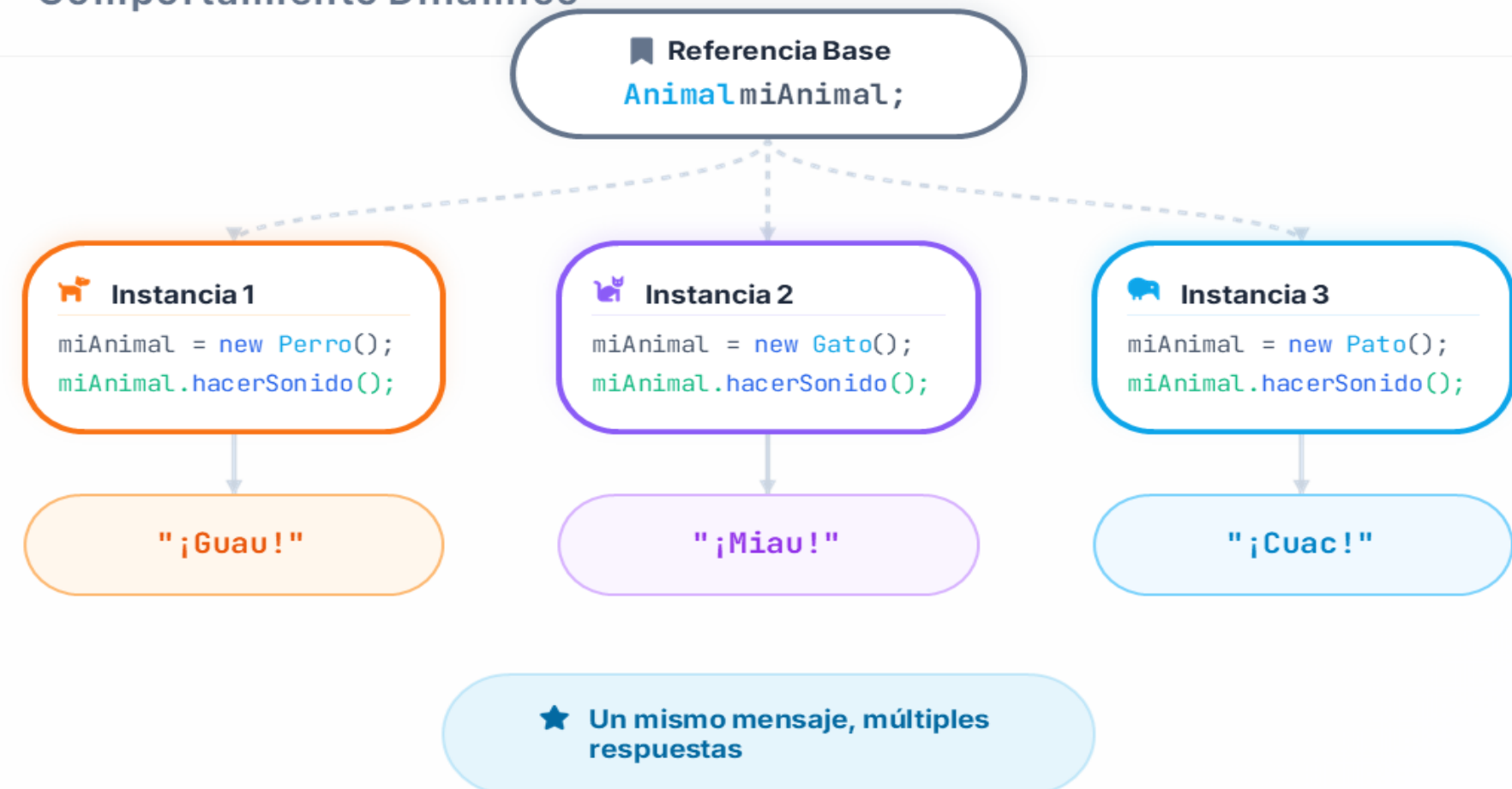
Ejecución Específica

Al invocar `hacerSonido()`, se ejecuta la versión sobrescrita del objeto actual.

Beneficio Clave:

- ✓ Código independiente de las implementaciones

Comportamiento Dinámico



Polimorfismo en colecciones

Manejando múltiples tipos desde una lista común

Listas de Abstracciones

El polimorfismo permite agrupar objetos de diferentes subclases en una **única colección** utilizando el tipo de la superclase.



Agrupación Unificada

List<Animal> puede contener Perro, Gato, Pato, etc.



Iteración Uniforme

Un solo bucle for-each procesa todos los elementos sin importar su tipo concreto.



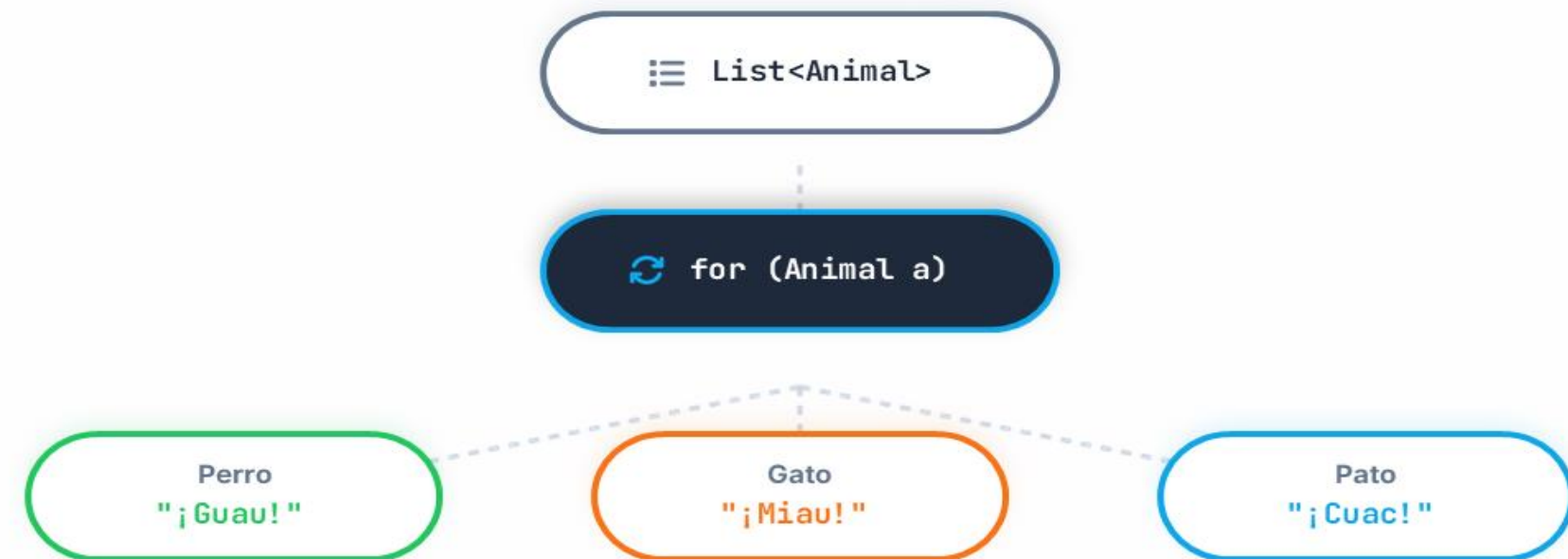
Ejecución Dinámica

Cada objeto responde de forma única al mismo mensaje (ej: hacerSonido).

Beneficio Clave:

- ✓ Simplifica el código y lo hace escalable

Flujo Polimórfico en Colecciones



```
List<Animal> animales = Arrays.asList( newPerro(), newGato(), newPato() );
for( Animal a : animales) {
    a.hacerSonido();
} // Dinámico según el objeto real
```



Métodos que reciben Animal

Desacoplamiento: Un solo método para múltiples tipos

Poder del Polimorfismo

Al usar la clase base como parámetro, un método puede operar con cualquier objeto que herede de ella, sin conocer su tipo específico.



Código Limpio

Evitas sobrecargar métodos repetitivos para cada tipo de animal.



Extensibilidad

Si añades una nueva clase (ej. Caballo), el método existente sigue funcionando intacto.



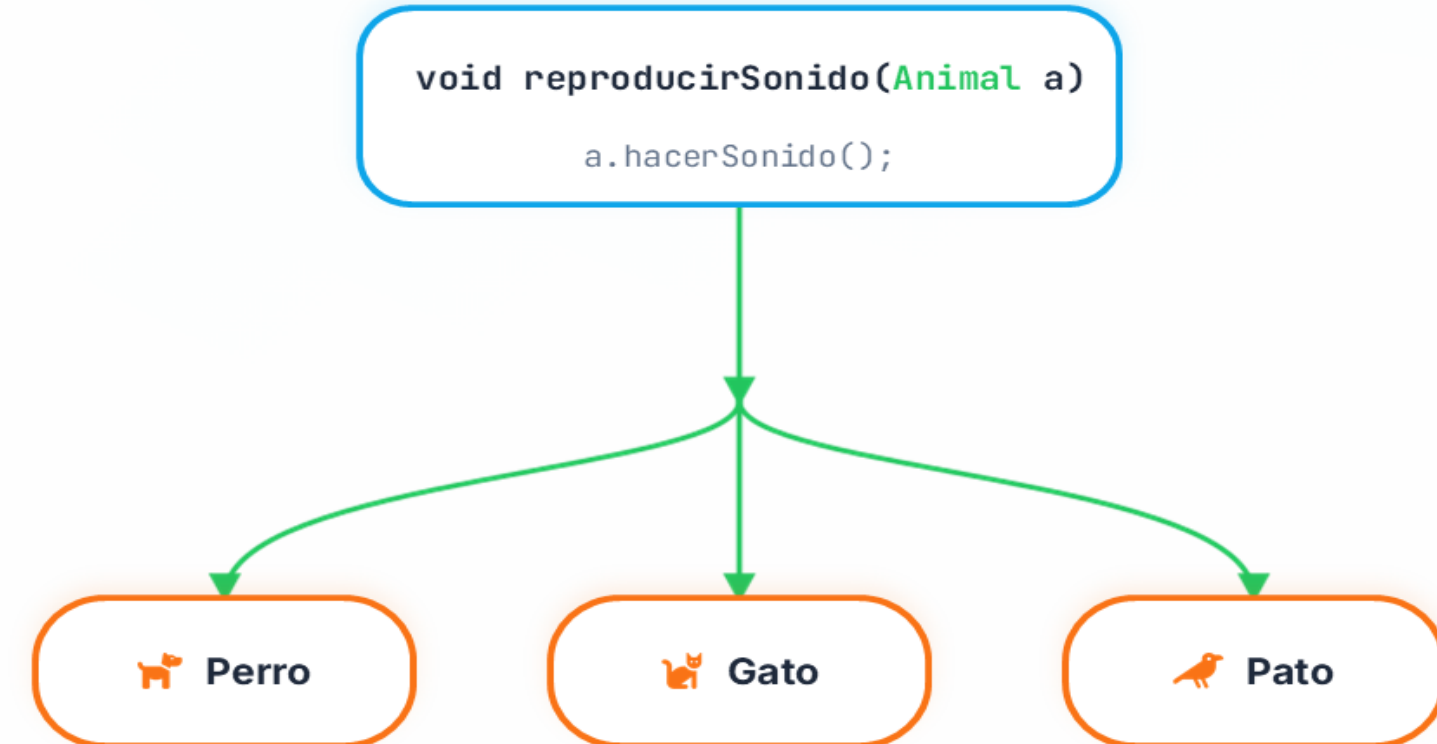
Desacoplamiento

El código cliente solo interactúa con la abstracción (Animal).

Beneficio Clave:

- ✓ Código Mantenable y Reutilizable

Flujo de Parámetro Polimórfico



Polimorfismo con Interfaces

Múltiples implementaciones de un mismo contrato de capacidad

Diseño por Contratos

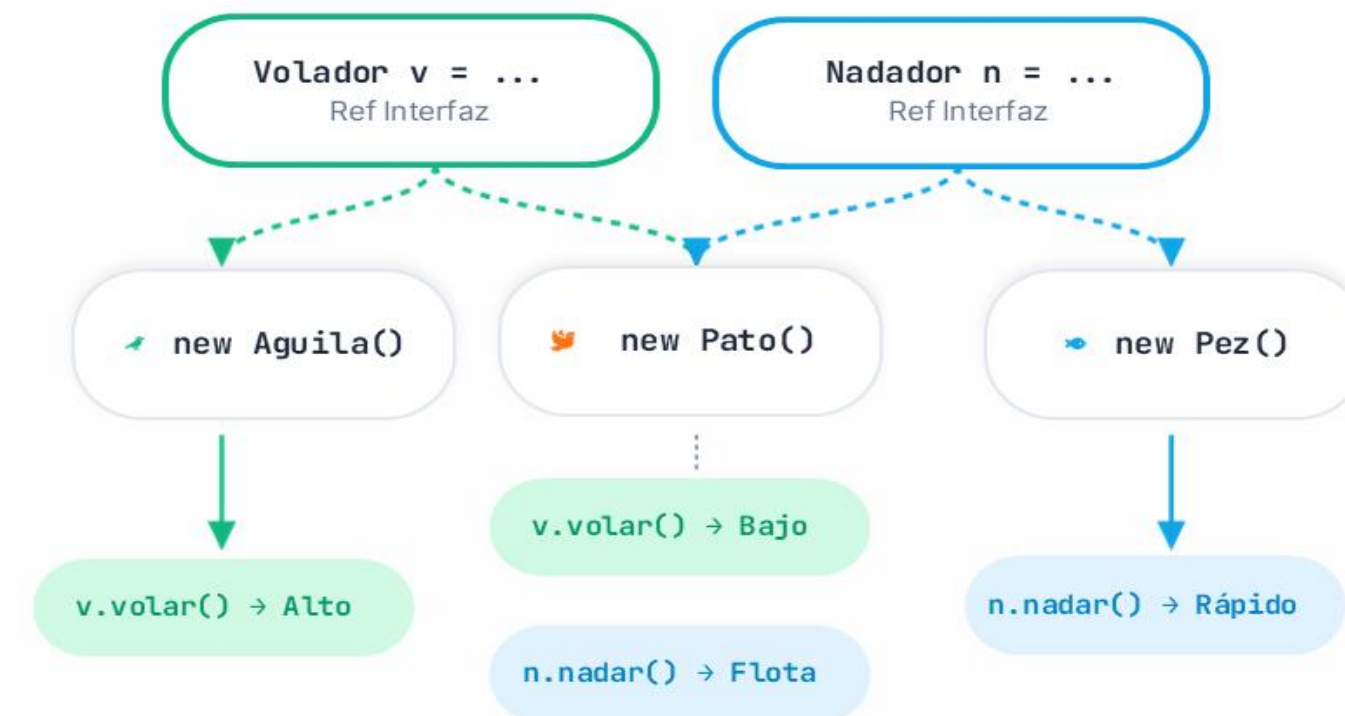
El polimorfismo de interfaces nos permite tratar objetos distintos basándonos en lo que **pueden hacer** (capacidades), no en lo que son.

Una referencia a `Volador` puede invocar `volar()` en cualquier clase que implemente ese contrato, independientemente de su jerarquía familiar.

Beneficios Clave:

-  Desacoplamiento extremo
-  Composición de comportamientos
-  Sistemas altamente modulares

Referencias y Capacidades



Buenas Prácticas al Usar Polimorfismo

Reglas para un código flexible y mantenible

✓ Reglas de Diseño

El verdadero poder del polimorfismo se alcanza cuando seguimos buenas prácticas de diseño orientado a objetos.



Programar vs Abstracciones

Usa tipos base (Animal, Volador) como variables, parámetros y retornos, no tipos concretos (Perro).



Sobrescribir con Sentido

Respetar el contrato original (Principio de Liskov). No lances excepciones inesperadas.



Evitar `instanceof`

El uso excesivo de comprobación de tipos es un "code smell" que indica mal uso del polimorfismo.



Mantener Contratos

Asegúrate de que cada implementación mantenga la lógica del método original.

Beneficio Clave:

🏆 **Sistemas Desacoplados y Escalables**

Checklist Visual



Programar vs Abstracciones

Usar `Animal a = new Perro()`



Acoplamiento fuerte

Usar `Perro p = new Perro()` si basta `Animal`



Delegar en el tipo real

Llamar directamente a `hacerSonido()`



Abusar de `instanceof`

Usar `if` para chequear el tipo de animal



Diseño Flexible y Escalable





Errores comunes y checklist

Prácticas clave para un polimorfismo robusto

⚠ Antipatrones a Evitar

Evita estos errores comunes para no comprometer los beneficios del diseño polimórfico.



Abusar de Condicionales

Usar `instanceof` en exceso en lugar de sobrescribir métodos.



Casts Innecesarios

Forzar tipos con casts cuando la interfaz común debería bastar.



Romper Contratos

Subclases que cambian drásticamente el propósito de un método.



Mezclar Responsabilidades

Diseñar clases base que intentan hacer de todo a la vez.

Resultado:

✔ Código Mantenable

Checklist de Polimorfismo



Consejo: Programa siempre hacia abstracciones (interfaces o clases abstractas) para maximizar la flexibilidad.



Conclusiones y Próximos Pasos

Cierre de Polimorfismo en Java

✓ Síntesis Final

El **polimorfismo** permite que un mismo mensaje sea respondido de múltiples formas, reduciendo el acoplamiento y mejorando la extensibilidad.



Abstracción

Depender de Animal o interfaces, no de Perro o Gato.



Extensibilidad

Añade nuevas clases sin modificar el código que las usa.



Ejecución Dinámica

Java decide qué método invocar en tiempo de ejecución.

Resultado:

✓ Código Limpio, Flexible y Escalable

Visión General del Ecosistema



→ Próximos Pasos

1 Practicar Colecciones

2 Principios SOLID

3 Patrón Strategy

Felicitaciones
¡Has finalizado el módulo!

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO

 Comisión Nacional
de Acreditación
CNA-Chile

NIVEL DE EXCELENCIA
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la
calidad / Vinculación con el Medio / Investigación, creación y/o innovación